

Peer-Review Critique: Sequential Training Plan for Physics-Informed Machine Learning and Building Performance Simulation in Civil Engineering Education

Executive Summary

This peer-review critique evaluates a seven-month Sequential Training Plan designed to train civil engineers in Physics-Informed Machine Learning (PIML) and Building Performance Simulation (BPS). The plan employs a "domain-first" pedagogical strategy, progressing from foundational EnergyPlus mastery (Months 1-3) through AI integration (Months 4-6) to autonomous agent development (Month 7). Drawing on recent literature (2023-2025), this critique examines four critical dimensions: (1) the feasibility of achieving EnergyPlus automation mastery within three months, (2) the validity of the domain-first learning strategy, (3) alignment with state-of-the-art PIML methods, and (4) the pedagogical logic of progressing from scripting to AI function calling.

The review finds strong theoretical support for the domain-first approach, with empirical evidence demonstrating that physics-informed models substantially reduce operational violations and improve data efficiency compared to purely data-driven methods. However, the three-month timeline for Phase 1 appears optimistic given limited empirical benchmarks and documented learner challenges. The curriculum demonstrates good alignment with emerging PIML methods (modularized neural networks, physics-informed reinforcement learning, symbolic networks) but would benefit from explicit coverage of recent advances in physics-informed graph neural networks and neuro-symbolic approaches. The scripting-to-AI progression is pedagogically sound but requires intermediate scaffolding steps—particularly prompt engineering, automated testing, and constraint validation—to ensure students can effectively validate AI-generated outputs against physical principles.

Key recommendations include: extending Phase 1 or requiring programming prerequisites, implementing mandatory domain checkpoints with physical consistency validation, explicitly incorporating recent PIML architectures (ModNN, KAN, physics-informed GNNs), and adding intermediate modules between scripting and autonomous agent development to address the gap between manual coding and AI-mediated automation.

Table of Contents

1. [Introduction](#)

2. [Critique Point 1: Feasibility of Phase 1 Timeline](#)

- 2.1 Evidence from Literature
- 2.2 Strengths Identified
- 2.3 Weaknesses and Gaps
- 2.4 Specific Recommendations

3. [Critique Point 2: Domain-First Strategy Validation](#)

- 3.1 Evidence from Literature
- 3.2 Strengths Identified
- 3.3 Weaknesses and Gaps
- 3.4 Specific Recommendations

4. [Critique Point 3: Curriculum Currency and State-of-the-Art Alignment](#)

- 4.1 Evidence from Literature
- 4.2 Strengths Identified
- 4.3 Weaknesses and Gaps
- 4.4 Specific Recommendations

5. [Critique Point 4: Pedagogical Logic of Scripting-to-AI Progression](#)

- 5.1 Evidence from Literature
- 5.2 Strengths Identified

- 5.3 Weaknesses and Gaps
- 5.4 Specific Recommendations

6. [Overall Assessment and Synthesis](#)

7. [Conclusion and Actionable Recommendations](#)

8. [References](#)

1. Introduction

The Sequential Training Plan under review represents an ambitious seven-month intensive program designed to transform civil engineers into AI-enabled building performance simulation specialists. The program's architecture reflects a deliberate pedagogical philosophy: students must first master the physical domain—building energy simulation, Python automation, and physics-informed machine learning theory—before being introduced to generative AI as an automation tool. This "domain-first" strategy culminates in Month 7 with the development of an autonomous researcher agent capable of conducting building performance analyses without human intervention.

The training plan is structured in three distinct phases. Phase 1 (Months 1-3) focuses on establishing deep competency in EnergyPlus simulation, Python-based automation at scale, and theoretical foundations of PIML. Phase 2 (Months 4-6) introduces generative AI capabilities progressively: from basic prompting and debugging, through retrieval-augmented generation (RAG) for technical documentation, to function calling that enables AI systems to invoke Python simulation tools. Phase 3 (Month 7) synthesizes these capabilities into a capstone project where students build an orchestrator agent that autonomously plans simulations, executes EnergyPlus runs, analyzes results, and generates reports grounded in Brazilian building standards (NBR 15.575).

This critique evaluates the plan against recent empirical and theoretical literature published between 2023 and 2025, a period marked by rapid advances in both physics-informed machine learning methods and the integration of large language models into engineering education. The analysis addresses four specific dimensions that are critical to the plan's success: the realism of the Phase 1 timeline, the empirical support for domain-first pedagogy, the currency of the PIML content relative to state-of-the-art methods, and the pedagogical soundness of the progression from manual scripting to AI-mediated automation. Each critique point is examined through the lens of recent literature, with attention to both supporting evidence and contradictory findings, followed by constructive recommendations grounded in empirical research.

2. Critique Point 1: Feasibility of Phase 1 Timeline

2.1 Evidence from Literature

The three-month timeline proposed for Phase 1—encompassing Python proficiency, EnergyPlus mastery, large-scale automation, and PIML theory—represents a significant learning demand. However, the recent literature provides limited direct empirical evidence on time-to-competency for EnergyPlus automation via Python libraries such as eppy and BESOS.

Maalek (2024) reports the most relevant empirical data from a multi-semester study integrating generative AI and problem-based learning into a Master's-level construction digitization curriculum. Across six semesters, 86.4% of students fully completed projects involving code generation for construction engineering management challenges, with an additional 13.6% achieving over 50% completion. Notably, this high completion rate was achieved with LLM assistance for code generation, suggesting that AI tools can accelerate skill acquisition. However, the study does not provide granular time-to-competency metrics for specific simulation tools or Python libraries, and the authors emphasize that "further research is needed as LLM capabilities evolve" (Maalek, 2024).

Liu (2024) examined LLM integration in system modeling and simulation courses, finding that LLMs assisted students in learning MATLAB programming and acquiring cross-disciplinary modeling knowledge. The study demonstrated improved understanding of complex concepts and increased student initiative, but again did not establish specific timelines for achieving automation competency in building simulation contexts.

The literature consistently documents several learner challenges that may extend the timeline beyond three months. Zakeri (2025) identified inconsistent procedural fidelity when dialogic AI is used as an intermediary, noting that "mandatory checkpoints and visual cues were inconsistently triggered, and question verbosity varied, raising fairness concerns." This variability suggests that AI-assisted learning, while potentially accelerating some aspects of skill development, can introduce unpredictability in learning outcomes without strict scaffolding. Zhang et al. (2024) similarly noted that students face challenges including "time constraints and knowledge gaps" when integrating new technologies into engineering curricula.

Chen et al. (2024) highlighted that domain knowledge gaps and data scarcity present significant barriers to practical model-building in building performance simulation education. Their work on component-based machine learning emphasized that students require robust understanding of building physics to effectively utilize ML methods, particularly when working with small or inconsistent datasets—a common scenario in educational settings.

2.2 Strengths Identified

The Phase 1 structure demonstrates several pedagogical strengths that may facilitate accelerated learning:

Progressive complexity and clear milestones. The plan structures learning in a logical sequence: basic Python and single simulations (Month 1), scaling to batch automation (Month 2), and theoretical synthesis (Month 3). Each month has concrete deliverables that provide clear assessment checkpoints.

Integration of modern tools. The explicit inclusion of eppy for IDF manipulation and BESOS for optimization workflows reflects awareness of current best practices in programmatic building simulation. The emphasis on "abandoning graphical interfaces" and mastering data manipulation aligns with professional practice in computational building science.

Theoretical grounding before practice. Month 3's focus on PIML theory before AI implementation (Phase 2) provides students with conceptual frameworks for understanding how physics can be integrated into machine learning models. This theoretical foundation may reduce cognitive load during later implementation phases.

Scaffolded automation. The progression from single simulations to 50-building variations with Latin Hypercube Sampling provides graduated exposure to automation complexity, allowing students to build confidence incrementally.

2.3 Weaknesses and Gaps

Despite these strengths, several significant weaknesses emerge from the literature review:

Absence of empirical time-to-competency benchmarks. The literature contains no studies establishing how long civil engineers with typical programming backgrounds require to achieve automation mastery of EnergyPlus via Python libraries. The three-month target appears to be based on instructional design intuition rather than empirical evidence. This represents a critical gap, as learning curves for complex simulation tools can vary substantially based on prior programming experience, mathematical background, and domain knowledge.

Underestimation of prerequisite variability. The plan does not explicitly specify required entry-level programming competencies. Alphas et al. (2024) noted the "challenge of developing and applying Generative AI models without a computer science background" for non-CS engineering students. If students enter Phase 1 with minimal Python experience, achieving the Month 2 deliverable (50-building automation with stochastic sampling) within the timeline may be unrealistic.

Insufficient scaffolding for automation complexity. While the plan progresses from 1 to 1,000 simulations conceptually, the jump from Month 1 (single simulation with plotting) to Month 2 (50 variations with LHS, JSON workflows, and BESOS integration) represents a substantial increase in technical complexity. Zakeri (2025) emphasized the importance of "instruction-based coverage checkpoints" and "single-question turns" to ensure consistent learning outcomes—elements not explicitly detailed in the Phase 1 structure.

Limited attention to debugging and error handling. Real-world EnergyPlus automation involves substantial debugging of IDF files, managing simulation failures, and validating physical coherence of results. The plan's deliverables focus on successful execution but do not explicitly address how students will develop troubleshooting skills within the compressed timeline.

Lack of formative assessment mechanisms. While each month has a deliverable, the plan does not specify intermediate checkpoints or formative assessments that would allow instructors to identify struggling students early and provide targeted support.

2.4 Specific Recommendations

Based on the literature evidence and identified gaps, the following recommendations are proposed:

Recommendation 1.1: Establish and enforce programming prerequisites. Require demonstrated proficiency in Python fundamentals (data structures, functions, file I/O, pandas basics) before Phase 1 begins, or extend Phase 1 to four months with Month 1 dedicated to intensive Python bootcamp for students lacking prerequisites. Alphas et al. (2024) demonstrated that hands-on projects with step-by-step guidance and incremental complexity are essential for non-CS engineering students.

Recommendation 1.2: Implement mandatory domain checkpoints with automated validation. Following Zakeri's (2025) recommendation for "instruction-based coverage checkpoints," introduce weekly graded checkpoints in Months 1-2 that automatically validate: (a) syntactic correctness of Python code, (b) successful EnergyPlus execution, and (c) physical coherence of simulation outputs (e.g., temperature bounds, energy balance checks). This addresses the "inconsistent procedural fidelity" problem documented in AI-assisted learning contexts.

Recommendation 1.3: Add intermediate scaffolding between Months 1 and 2. Introduce a "Month 1.5" mini-module (2 weeks) focused specifically on: (a) parametric IDF modification with eppy, (b) batch simulation management, and (c) error handling and debugging strategies. This reduces the cognitive leap between single-simulation mastery and large-scale automation.

Recommendation 1.4: Instrument the curriculum to collect time-to-competency data. Systematically track student progress through automated logging of: time spent on each deliverable, number of debugging iterations, and success rates on intermediate checkpoints. This empirical data will enable evidence-based refinement of the timeline in future iterations. As Maalek (2024) noted, "further research is needed" to understand how evolving AI capabilities affect learning timelines.

Recommendation 1.5: Integrate LLM-assisted learning with structured scaffolding. Following Maalek's (2024) successful integration of LLMs for code generation, allow students to use AI assistants (e.g., GitHub Copilot, ChatGPT) for debugging and code completion, but require: (a) documented prompts, (b) manual code review and explanation, and (c) unit tests for all AI-generated functions. This leverages AI acceleration while maintaining deep learning, addressing Liu's (2024) finding that LLMs can "facilitate code generation, which would otherwise be complex without additional formal training."

Recommendation 1.6: Provide differentiated pathways based on entry competencies. Offer an "accelerated track" for students with strong programming backgrounds and an "extended track" (4 months) for those requiring additional foundational support. This addresses the prerequisite variability problem while maintaining rigor for all students.

3. Critique Point 2: Domain-First Strategy Validation

3.1 Evidence from Literature

The domain-first pedagogical strategy—requiring mastery of building physics and simulation before introducing AI automation—finds substantial support in recent literature, though primarily from technical performance studies rather than controlled pedagogical experiments.

Empirical evidence of physics-informed superiority. Jiang et al. (2024) provide the most compelling evidence for the value of domain knowledge integration in their comparative study of modularized neural networks (ModNN) versus standard LSTM models for building control. Their results demonstrate dramatic differences in operational safety: "the LSTM model failed in control optimization, resulting in an 89°C-h temperature violation, whereas the ModNN showed only a 0.57°C-h violation and achieved up to a 78% peak load reduction" (Jiang et al., 2024). This 156-fold reduction in temperature violations illustrates that physics-aware architectural choices markedly reduce harmful control outcomes, even when pure prediction accuracy is similar. This finding directly supports the training plan's emphasis on understanding physical constraints before deploying AI systems.

Chen et al. (2024) demonstrated that incorporating semantic domain knowledge through Component-Based Machine Learning (CBML) significantly improves model robustness with extremely small datasets (1% to 0.0125% sampling rates). Their approach showed three key advantages: "improved robustness with extremely small/inconsistent datasets, efficient use

of diverse records, and handling incomplete data with high interpretability and reduced training time" (Chen et al., 2024). This evidence suggests that students who develop strong domain knowledge first will be better equipped to work effectively in data-limited regimes common in building performance applications.

Saeed et al. (2023) compared physics-informed model-based reinforcement learning (PI-MBRL) against vanilla deep Q-networks (v-DQN) for building heating control. Their results showed that PI-MBRL "outperforms the v-DQN when trained on just 4 weeks of heating data of the building and provides greater thermal comfort when tested on two entire months" (Saeed et al., 2023). This demonstrates that injecting physical knowledge improves performance in low training data regimes—a critical consideration for educational contexts where students may not have access to extensive datasets.

Theoretical frameworks supporting knowledge integration. Chen et al. (2023) proposed a three-level paradigm for systematic incorporation of prior knowledge into ML pipelines, emphasizing the pedagogical value of building domain competence before or alongside ML instruction. Their framework suggests that "no one best way to represent knowledge or formalize information" exists, implying that students need deep domain understanding to make informed choices about knowledge representation in PIML systems.

Kuo et al. (2023) developed a physics-informed framework for modeling buildings from incomplete information, demonstrating how domain knowledge enables practitioners to generate comprehensive building energy models even with sparse data. This capability is particularly relevant for civil engineers who often face incomplete building documentation in retrofit and assessment projects.

3.2 Strengths Identified

The domain-first strategy demonstrates several evidence-based strengths:

Prevention of catastrophic failures. The Jiang et al. (2024) findings on temperature violations provide strong justification for ensuring students understand physical constraints before deploying AI systems. The 89°C-h violation in the LSTM model represents a failure mode that could have serious consequences in real building applications. By mastering domain knowledge first, students develop the ability to recognize and prevent such failures.

Enhanced data efficiency. Multiple studies (Chen et al., 2024; Saeed et al., 2023) demonstrate that physics-informed approaches require substantially less training data than purely data-driven methods. For educational contexts where data collection is resource-intensive, this efficiency is pedagogically valuable. Students who understand the physics can achieve better results with smaller datasets, making projects more feasible within academic constraints.

Improved interpretability and debugging capability. Chen et al.'s (2024) emphasis on interpretability suggests that domain-first learning enables students to understand and debug model behavior more effectively. When students know what physically plausible outputs should look like, they can more readily identify when models produce nonsensical results.

Foundation for informed automation decisions. The training plan's rationale that students must understand "what they're automating before learning how to automate it" aligns with the literature's emphasis on informed decision-making. Students who manually perform simulations, generate data, and analyze results are better positioned to recognize which processes benefit most from automation and which require human oversight.

3.3 Weaknesses and Gaps

Despite strong technical evidence, several pedagogical gaps limit the strength of the domain-first validation:

Absence of controlled pedagogical trials. The literature contains no randomized controlled trials or quasi-experimental studies comparing domain-first versus concurrent domain+AI learning sequences in educational settings. All supporting evidence comes from technical performance comparisons (physics-informed vs. purely data-driven models) rather than pedagogical experiments. This represents a significant evidence gap: while we know physics-informed models perform better, we do not have direct evidence that teaching domain knowledge first produces better learning outcomes than integrated approaches.

Potential for extended time-to-capability. The domain-first approach necessarily delays students' exposure to AI tools until Month 4. Maalek (2024) and Liu (2024) demonstrated that LLMs can assist students in generating working code "without deep prior training," suggesting that some students might benefit from earlier AI exposure. The strict sequencing may unnecessarily extend the time until students can leverage AI assistance for learning domain concepts.

Risk of motivation loss during extended foundational phase. The plan requires three months of intensive simulation work before students encounter the "exciting" AI components. Educational psychology research (not directly represented in this corpus) suggests that delayed gratification can reduce student motivation, particularly for learners attracted to the program specifically for AI content.

Limited evidence on optimal integration timing. Chen et al. (2023) noted that "no one best way to represent knowledge or formalize information" exists, suggesting that rigid sequencing may not be universally optimal. Some students might benefit from iterative exposure—learning basic domain concepts, applying AI tools, encountering failures, and returning to deepen domain knowledge in response to those failures.

Insufficient attention to AI-assisted domain learning. The plan treats AI as purely an automation tool introduced in Phase 2, but recent literature (Liu, 2024; Maalek, 2024) demonstrates that LLMs can effectively support domain learning itself. Students could potentially use AI assistants to understand EnergyPlus documentation, debug simulation errors, and explore "what-if" scenarios during Phase 1, accelerating domain mastery.

3.4 Specific Recommendations

Recommendation 2.1: Maintain domain-first core while introducing AI as learning support. Preserve the fundamental sequence (domain mastery before autonomous AI deployment) but allow students to use LLMs as learning assistants during Phase 1 for: (a) explaining EnergyPlus documentation, (b) debugging code errors, and (c) generating exploratory code snippets. Require students to validate all AI-generated explanations against official documentation and to manually review all code. This approach leverages Liu's (2024) finding that LLMs improve understanding while maintaining the domain-first principle.

Recommendation 2.2: Implement early "preview" modules to maintain motivation. Add a 2-hour "AI Preview" session in Month 1 where students see a demonstration of the Month 7 autonomous agent, understand the end goal, and appreciate why domain knowledge is essential for building such systems. Include examples of AI failures due to lack of physics constraints (e.g., the 89°C-h violation from Jiang et al., 2024) to motivate the domain-first approach.

Recommendation 2.3: Add mandatory physical consistency validation throughout Phase 2. When AI tools are introduced in Months 4-6, require students to validate all AI-generated outputs against physical principles learned in Phase 1. Create a "Physics Validation Checklist" that students must complete for every AI-assisted task, explicitly connecting Phase 2 activities back to Phase 1 knowledge. This reinforces the domain-first rationale and prevents students from treating AI as a "black box."

Recommendation 2.4: Conduct pedagogical research to validate the approach. Design and implement a quasi-experimental study comparing cohorts trained with the domain-first sequence versus an integrated approach where domain and AI concepts are taught concurrently. Measure outcomes including: (a) ability to identify physically implausible model outputs, (b) debugging effectiveness, (c) final project quality, and (d) long-term retention of domain knowledge. This addresses the critical evidence gap identified in the literature review.

Recommendation 2.5: Develop explicit "bridging" content between Phases 1 and 2. Create a transition module at the start of Month 4 that explicitly connects domain knowledge from Phase 1 to AI applications in Phase 2. Include exercises where students: (a) identify which physical constraints should be encoded in AI systems, (b) design validation tests based on domain knowledge, and (c) analyze case studies of AI failures due to insufficient physics integration (using examples from Jiang et al., 2024; Saeed et al., 2023).

Recommendation 2.6: Incorporate iterative deepening rather than strict sequencing. Consider a spiral curriculum approach where students encounter domain concepts, apply AI tools, experience failures, and return to deepen domain understanding. For example, introduce basic AI prompting in Month 2 for debugging assistance, observe where students struggle due to insufficient domain knowledge, and use those struggles as motivation for deeper domain study in Month 3.

4. Critique Point 3: Curriculum Currency and State-of-the-Art Alignment

4.1 Evidence from Literature

The training plan cites several recent methodologies (Ma et al., 2024; Jiang et al., 2025; Yang et al., 2024; Wang et al., 2025) in Month 3's PIML theory module, suggesting awareness of current research. However, a systematic comparison against the 2023-2025 literature reveals both strong alignments and notable gaps.

Well-represented state-of-the-art methods. The curriculum's emphasis on physics-informed approaches aligns well with several cutting-edge developments documented in recent literature:

Jiang et al. (2024, 2025) introduced modularized neural networks (ModNN) that explicitly measure and enforce physical consistency, demonstrating superior control safety compared to standard deep learning approaches. Their work on "Physics-informed Modularized Neural Network for Advanced Building Control by Deep Reinforcement Learning" represents exactly the type of integration the training plan aims to teach—combining domain knowledge with advanced AI methods for operational building control (Jiang et al., 2025).

Chen et al. (2024) proposed Kolmogorov-Arnold Networks (KAN) and symbolic neural networks for building physics applications, demonstrating that "KANs show superior adaptability in data-sparse environments, outperforming MLPs in R^2 scores across most subsample rates and tasks." Their work showed that prior knowledge integration significantly improved model performance, with R^2 increasing from 0.20 to 0.99 in one case study. The paper explicitly recommends that "curriculum design should integrate domain knowledge to guide machine learning models" (Chen et al., 2024), directly supporting the training plan's approach.

Ma et al. (2024) developed physics-informed ensemble learning with residual modeling for building energy prediction, representing recent (2024) directions for combining ensemble methods with residual physics modeling. This work exemplifies the type of hybrid approaches that students should understand.

Shan et al. (2025) introduced physics-informed and explainable graph neural networks (GNNs) for urban building energy modeling, demonstrating that "models explicitly incorporating interpretable physical edge features, such as inter-building

distance and angular relation, achieve significantly improved prediction accuracy and robustness over standard approaches" (Shan et al., 2025). Their work on GraphSAGE and physics-informed GAT variants represents an emerging frontier in scalable urban energy modeling.

Danish et al. (2025) proposed physics-guided memory networks for building energy modeling, published in Energy and AI, representing another recent architectural innovation that combines domain knowledge with advanced neural architectures.

Emerging methods with limited or no coverage. Several important recent developments appear underrepresented or absent from the training plan:

Ashayeri et al. (2024) demonstrated hybrid physics-based machine learning for integrated energy and exposure modeling, achieving "up to 0.96 R-squared accuracy using ANNs, accelerating simulation 200 times compared to conventional physics-based methods." Their integration of EnergyPlus with CONTAM and CFD for indoor air quality represents an important co-simulation approach not explicitly addressed in the curriculum.

Labib (2024) applied physics-informed neural networks specifically to daylighting simulations, representing a specialized application domain that could enrich the curriculum's scope beyond thermal and energy performance.

The literature review revealed no explicit discussion of foundation models or large-scale pre-trained models for building performance applications in the training plan. While Phase 2 introduces Gemini Pro, the curriculum does not address how foundation models might be fine-tuned or adapted for building-specific tasks—an emerging area of research.

Physics-guided generative models (diffusion models, physics-constrained generative priors) are not evidenced in the training plan, though these represent cutting-edge directions in scientific machine learning more broadly.

4.2 Strengths Identified

Strong coverage of core PIML paradigms. The training plan's Month 3 reading list includes several seminal recent works that establish students' understanding of fundamental PIML concepts: physics-constrained modeling, modular architectures, and domain-specific applications. This provides a solid theoretical foundation.

Emphasis on practical tools and workflows. The explicit inclusion of eppy, BESOS, and ACCIM Library demonstrates attention to the practical toolchain required for professional building performance work. The focus on JSON-Python workflows (citing Osei-Owusu et al., 2025) and surrogate optimization pipelines (Kubwimana & Najafi, 2023) reflects awareness of modern computational workflows.

Integration of recent applications. The inclusion of tropical retrofitting applications (Wang et al., 2025) and thermal resilience (Mehraban et al.) demonstrates that the curriculum addresses contemporary climate adaptation challenges, not just theoretical PIML concepts.

Cloud-based ML infrastructure. The use of Google Cloud Vertex AI, Gemini Pro, and associated MLOps tools positions students to work with industry-standard cloud platforms rather than only local computational resources. This aligns with professional practice in 2024-2025.

4.3 Weaknesses and Gaps

Absence of graph neural network methods. Despite Shan et al.'s (2025) demonstration that physics-informed GNNs achieve superior generalization for urban-scale building energy modeling, the training plan does not explicitly include graph-based methods. Given the importance of spatial relationships in building performance (district-scale analysis, urban heat islands, neighborhood effects), this represents a significant gap for students who may work on urban-scale projects.

Limited coverage of neuro-symbolic approaches. While Chen et al. (2024) demonstrated that KANs and symbolic neural networks can "rediscover equations and augment interpretability in building physics," the training plan does not explicitly address symbolic regression or equation discovery methods. These approaches are particularly valuable for educational contexts because they produce interpretable models that students can validate against known physics.

Insufficient attention to co-simulation frameworks. Ashayeri et al.'s (2024) work on integrating EnergyPlus with CONTAM and CFD represents an important capability for holistic building performance assessment (energy + indoor air quality + airflow). The training plan focuses exclusively on EnergyPlus, potentially limiting students' ability to address multi-domain problems.

Lack of explicit coverage of model interpretability and explainability. While several cited papers emphasize interpretability (Chen et al., 2024; Shan et al., 2025), the training plan does not include explicit modules on explainable AI methods, feature importance analysis, or sensitivity analysis—all critical for building trust in AI-driven building performance predictions.

Unclear coverage of recent LLM capabilities for engineering. The training plan introduces Gemini Pro for function calling but does not address recent advances in code generation, multi-modal reasoning, or tool-augmented LLMs that could significantly enhance building performance workflows. Maalek (2024) and Liu (2024) demonstrated substantial benefits from LLM integration in engineering education, but the training plan's treatment of LLMs appears limited to specific use cases (debugging, RAG, function calling) rather than exploring their full potential.

Missing empirical validation of cited sources. The training plan cites "Osei-Owusu et al. (2025)" for modular JSON-Python workflows, but this source does not appear in the literature corpus reviewed for this critique. This raises questions about whether all cited methodologies have been rigorously validated or whether some represent unpublished or preliminary work.

4.4 Specific Recommendations

Recommendation 3.1: Add explicit module on physics-informed graph neural networks. Incorporate a 1-week sub-module in Month 3 covering graph-based representations of building systems and urban energy networks. Include hands-on exercises using Shan et al.'s (2025) GraphSAGE approach for district-scale energy modeling. This addresses the spatial reasoning gap and prepares students for urban-scale applications.

Recommendation 3.2: Integrate neuro-symbolic methods and equation discovery. Add content on Kolmogorov-Arnold Networks (KAN) and symbolic regression to Month 3, using Chen et al.'s (2024) framework. Include exercises where students use symbolic methods to rediscover known building physics equations (e.g., heat transfer relationships) from simulation data. This reinforces domain knowledge while teaching cutting-edge methods. Chen et al. (2024) provide a decision tree (Figure 4) that can serve as a pedagogical tool for guiding students in selecting appropriate methods based on data availability and problem complexity.

Recommendation 3.3: Expand tool coverage to include co-simulation frameworks. Add a 1-week module in Month 2 on co-simulation, introducing CONTAM for airflow and demonstrating EnergyPlus-CONTAM coupling following Ashayeri et al.'s (2024) approach. This prepares students for integrated energy and indoor environmental quality assessments, addressing the growing importance of healthy buildings.

Recommendation 3.4: Create dedicated module on explainable AI for building performance. Add explicit content in Month 6 on interpretability methods: SHAP values, feature importance analysis, partial dependence plots, and physics-based validation of learned relationships. Require students to explain and justify AI model predictions using domain knowledge, connecting back to Phase 1 learning.

Recommendation 3.5: Update LLM content to reflect 2024-2025 capabilities. Expand Phase 2 to include: (a) multi-modal reasoning with vision-language models for analyzing building drawings and thermal images, (b) tool-augmented LLMs that can autonomously search documentation and execute code, and (c) fine-tuning approaches for domain-specific applications. This aligns with the rapid evolution of LLM capabilities documented by Maalek (2024) and Liu (2024).

Recommendation 3.6: Verify and update all cited sources. Conduct a systematic review of all cited methodologies to ensure they are published, peer-reviewed, and accessible to students. Replace any unpublished or inaccessible sources with validated alternatives from the 2023-2025 literature. For example, if "Osei-Owusu et al. (2025)" is not yet published, substitute with documented JSON-Python workflow examples from the BESOS or Eppy documentation.

Recommendation 3.7: Add "State-of-the-Art Review" assignment in Month 3. Require students to conduct a mini literature review of one recent PIML method (published 2024-2025) not explicitly covered in the curriculum, present it to the class, and propose how it could be applied to a building performance problem. This ensures students develop the skill of staying current with rapidly evolving methods and prevents the curriculum from becoming outdated.

Recommendation 3.8: Incorporate physics-informed reinforcement learning explicitly. While the training plan mentions deep reinforcement learning in Month 7, it should explicitly cover physics-informed model-based RL following Saeed et al.'s (2023) approach. This is particularly relevant for the autonomous agent capstone, where students need to understand how to inject physical constraints into RL-based control policies.

5. Critique Point 4: Pedagogical Logic of Scripting-to-AI Progression

5.1 Evidence from Literature

The training plan's progression from manual scripting (Months 1-2) to AI function calling (Month 6) and autonomous agents (Month 7) reflects a scaffolded learning approach. Recent literature provides mixed evidence on the necessity and effectiveness of this sequence.

Evidence supporting scripting-first approaches. Maalek (2024) implemented a blended model integrating generative AI with problem-based learning across six semesters, achieving 86.4% full project completion. Critically, the study notes that LLMs "facilitated code generation, which would otherwise be complex without additional formal training," but emphasizes that "further research is needed as LLM capabilities evolve" (Maalek, 2024). This suggests that while LLMs can compensate for limited coding skills, the optimal balance between manual coding instruction and AI assistance remains an open question.

Liu (2024) found that LLMs "assisted students in learning MATLAB programming tasks, allowing students to learn MATLAB commands and programming techniques more conveniently" and helped students acquire "cross-disciplinary knowledge related to modeling and simulation." However, the study emphasized that LLMs augmented rather than replaced programming instruction, suggesting that foundational coding skills remain valuable for deeper understanding.

Alphinas et al. (2024) designed a learning environment for non-CS engineering students that explicitly addresses both coding and generative AI prompting, noting the "challenge of developing and applying Generative AI models without a computer science background." Their framework implies that programming foundations remain valuable for managing risks and deepening understanding, even when AI tools are available.

Evidence suggesting earlier AI integration may be beneficial. Maalek's (2024) finding that LLMs enabled students to complete complex projects "without additional formal training" suggests that strict sequencing may not be universally necessary. Some students might benefit from using AI assistance during the scripting phase itself, potentially accelerating skill acquisition.

Zhang et al. (2024) reported that integrating generative AI "improved student project quality, depth, and originality after two semesters, fostering lifelong learning." The study noted that AI helped address "challenges like time constraints and knowledge gaps," suggesting that earlier AI introduction might help struggling students overcome barriers during the foundational phase.

Evidence on required intermediate steps. Zakeri (2025) identified critical gaps in AI-assisted learning that require explicit pedagogical attention: "mandatory checkpoints and visual cues were inconsistently triggered, and question verbosity varied, raising fairness concerns." The study recommends "instruction-based coverage checkpoints, single-question turns, and timed multimodal prompts" to ensure consistent learning outcomes. This suggests that the jump from scripting to autonomous AI agents requires intermediate scaffolding.

The physics-informed modeling literature (Jiang et al., 2024; Chen et al., 2024) emphasizes the importance of constraint validation and physical consistency checking. Jiang et al.'s (2024) demonstration that LSTM models can produce 89°C-h temperature violations while ModNN produces only 0.57°C-h violations highlights the critical need for students to validate AI outputs against physical principles—a skill that requires both domain knowledge and systematic validation procedures.

5.2 Strengths Identified

Clear pedagogical rationale. The progression from manual scripting to AI automation ensures students understand the underlying tasks before delegating them to AI systems. This aligns with constructivist learning theory and the principle that students should understand "what" before learning "how to automate."

Graduated complexity. The sequence provides clear stepping stones: basic scripting (Month 1-2), AI prompting for debugging (Month 4), RAG for documentation (Month 5), function calling (Month 6), and full orchestration (Month 7). Each step builds on previous capabilities, reducing cognitive overload.

Emphasis on validation capabilities. By requiring students to manually perform simulations and analyze results in Phase 1, the plan ensures they develop the ability to recognize physically implausible outputs—a critical skill for validating AI-generated results in Phase 2.

Integration of modern AI capabilities progressively. The plan introduces AI capabilities in a logical order: prompting → retrieval → tool use → orchestration. This mirrors the actual capabilities of modern LLM systems and prepares students for professional practice.

5.3 Weaknesses and Gaps

Lack of intermediate validation and testing modules. The plan jumps from basic function calling (Month 6) to full autonomous agents (Month 7) without explicit instruction on: (a) unit testing AI-generated code, (b) integration testing for multi-step workflows, (c) constraint validation frameworks, or (d) error handling and recovery strategies. Zakeri's (2025) emphasis on "mandatory checkpoints" suggests these intermediate steps are critical for reliable AI systems.

Insufficient coverage of prompt engineering as a skill. While Month 4 introduces prompting for debugging, the plan does not treat prompt engineering as a systematic skill requiring practice and refinement. Maalek (2024) noted the importance of teaching "effective LLM interaction," but the training plan provides only one deliverable (fixing a bug with Gemini) to develop this skill.

Missing content on AI failure modes and limitations. The plan does not explicitly address: (a) hallucination detection and mitigation, (b) understanding when AI assistance is appropriate versus when manual coding is necessary, or (c) recognizing the boundaries of AI capabilities. Zhang et al. (2024) emphasized the need for "meticulous attention to ethical considerations," but the training plan's treatment of AI risks appears limited.

Unclear connection between RAG (Month 5) and function calling (Month 6). The plan treats these as separate capabilities, but modern agentic systems integrate retrieval and tool use. Students may struggle to understand how to combine these capabilities in Month 7 without explicit instruction on their integration.

Limited attention to debugging AI-generated code. While Month 4 uses AI for debugging human-written code, the plan does not address the reverse: how to debug AI-generated code. This is a critical skill for Month 7's autonomous agent, where students must troubleshoot AI-generated simulation workflows.

Absence of human-in-the-loop design principles. The Month 7 autonomous agent is described as operating "without human intervention," but professional practice increasingly emphasizes human-AI collaboration rather than full automation. The plan does not address when and how to design appropriate human oversight mechanisms.

5.4 Specific Recommendations

Recommendation 4.1: Add "Month 5.5" module on prompt engineering and AI testing. Insert a 2-week intensive module between Months 5 and 6 covering: (a) systematic prompt engineering techniques (few-shot learning, chain-of-thought, role-based prompting), (b) automated testing of AI-generated outputs, (c) unit testing frameworks for AI-generated code, and (d) constraint validation methods. This addresses the gap between basic AI use and autonomous agent development.

Recommendation 4.2: Require AI-assisted learning during Phase 1 with mandatory validation. Allow students to use LLMs as coding assistants during Months 1-2, but require: (a) documented prompts for all AI interactions, (b) manual code review and line-by-line explanation of AI-generated code, (c) unit tests for all AI-generated functions, and (d) physical validation of simulation results. This leverages Maalek's (2024) finding that LLMs can accelerate learning while maintaining the domain-first principle through mandatory validation.

Recommendation 4.3: Expand Month 4 to include systematic prompt engineering curriculum. Transform Month 4 from a single debugging exercise into a comprehensive prompt engineering module covering: (a) prompt design patterns for different tasks (code generation, debugging, explanation, documentation), (b) iterative prompt refinement, (c) evaluating AI response quality, and (d) recognizing hallucinations and errors. Include multiple graded exercises with rubrics assessing prompt effectiveness.

Recommendation 4.4: Add explicit module on physical constraint validation in Month 6. When introducing function calling, require students to implement automated validation checks that verify AI-generated simulation parameters against physical constraints. Use examples from Jiang et al. (2024) showing catastrophic failures (89°C-h violations) to motivate the importance of validation. Require students to build a "Physics Validation Library" that checks: temperature bounds, energy balance, mass conservation, and other domain-specific constraints.

Recommendation 4.5: Redesign Month 7 to emphasize human-AI collaboration rather than full automation. Reframe the autonomous agent as a "human-in-the-loop decision support system" where the AI proposes simulation plans and analyses, but requires human approval at critical decision points. This aligns better with professional practice and addresses ethical concerns about fully autonomous systems. Require students to explicitly design: (a) which decisions require human oversight, (b) what information the AI must provide to support human decision-making, and (c) how to handle cases where AI confidence is low.

Recommendation 4.6: Add intermediate "AI-assisted workflow" projects in Months 4-5. Before attempting full autonomous agents, require students to complete 2-3 smaller projects where they use AI to assist with specific workflow components: (a) AI-assisted parametric study design (Month 4), (b) AI-assisted result interpretation and visualization (Month 5), (c) AI-assisted error diagnosis and debugging (Month 5). Each project should require students to validate AI outputs against domain knowledge.

Recommendation 4.7: Incorporate explicit instruction on debugging AI-generated code. Add content in Month 6 on: (a) common errors in AI-generated simulation code, (b) strategies for diagnosing AI-generated bugs, (c) when to regenerate versus manually fix AI code, and (d) how to provide effective feedback to AI systems to improve subsequent generations. This prepares students for the inevitable debugging required in Month 7.

Recommendation 4.8: Create "AI Failure Case Studies" throughout Phase 2. In each month of Phase 2, include analysis of real-world cases where AI systems failed in engineering contexts due to: lack of domain knowledge, insufficient validation, hallucinations, or inappropriate automation. Use Jiang et al.'s (2024) LSTM failure case as a recurring example. Require students to propose how domain knowledge and validation procedures could have prevented each failure.

6. Overall Assessment and Synthesis

The Sequential Training Plan represents an ambitious and theoretically well-grounded approach to preparing civil engineers for the intersection of building performance simulation and artificial intelligence. The plan's core strengths lie in its domain-first pedagogical philosophy, progressive complexity, and integration of cutting-edge PIML methods. The recent literature (2023-2025) provides substantial technical validation for the plan's emphasis on physics-informed approaches, with compelling evidence that domain knowledge integration dramatically improves model safety, data efficiency, and interpretability.

However, the critique reveals several systematic challenges that may impede successful implementation:

Timeline feasibility concerns. The three-month Phase 1 timeline appears optimistic given the absence of empirical time-to-competency benchmarks in the literature and documented learner challenges including knowledge gaps, time pressure, and automation complexity. The plan would benefit from either extending Phase 1, requiring stronger programming prerequisites, or implementing more intensive scaffolding and formative assessment.

Evidence gaps in pedagogical sequencing. While technical literature strongly supports physics-informed methods over purely data-driven approaches, there is no direct empirical evidence from controlled pedagogical studies that domain-first teaching produces better learning outcomes than integrated approaches. The plan's strict sequencing may be pedagogically sound, but this remains an assumption rather than an evidence-based conclusion. Recent studies (Maalek, 2024; Liu, 2024) suggest that LLMs can effectively support learning even without deep prior training, raising questions about whether earlier AI integration might accelerate rather than impede domain mastery.

Curriculum currency strengths and gaps. The plan demonstrates good awareness of recent PIML developments (ModNN, physics-informed RL, component-based ML) but has notable gaps in emerging areas including physics-informed graph neural networks, neuro-symbolic methods, co-simulation frameworks, and explainable AI. The rapid evolution of LLM capabilities (documented by Maalek, 2024) also suggests that the plan's treatment of generative AI may quickly become outdated without mechanisms for continuous curriculum updating.

Insufficient intermediate scaffolding. The most significant pedagogical weakness is the lack of intermediate steps between manual scripting (Months 1-2) and autonomous agent development (Month 7). The literature consistently emphasizes the need for mandatory checkpoints, systematic validation procedures, and graduated exposure to AI capabilities. The plan's current structure risks leaving students unprepared for the complexity of Month 7's capstone project.

Synthesis of recommendations. The critique's recommendations cluster into four strategic priorities:

1. Strengthen Phase 1 foundations through programming prerequisites, extended timelines for students lacking prerequisites, mandatory domain checkpoints with automated validation, and empirical tracking of time-to-competency.
2. Refine the domain-first approach by allowing AI-assisted learning during Phase 1 (with mandatory validation), adding motivational "preview" content, implementing explicit bridging between phases, and conducting pedagogical research to validate the sequencing hypothesis.
3. Update curriculum content to include physics-informed GNNs, neuro-symbolic methods, co-simulation frameworks, explainable AI, and expanded LLM capabilities, while verifying all cited sources and adding mechanisms for continuous updating.
4. Add intermediate scaffolding between scripting and autonomous agents through dedicated modules on prompt engineering, AI testing and validation, physical constraint checking, debugging AI-generated code, and human-AI collaboration principles.

Comparative assessment. Relative to typical engineering AI education programs, this training plan is notably ambitious in its integration depth and domain specificity. Most programs either teach AI methods abstractly (without deep domain integration) or teach domain skills without modern AI capabilities. The plan's attempt to deeply integrate both represents a valuable innovation. However, this ambition also creates implementation risks: the compressed timeline, high cognitive load, and need for instructors with expertise spanning building physics, Python automation, PIML theory, and modern LLM capabilities may limit scalability and replicability.

Contextual considerations. The plan's success will likely depend heavily on: (a) student entry competencies, particularly programming background; (b) instructor expertise across the full technical stack; (c) computational infrastructure (cloud credits, simulation licenses); (d) class size (intensive mentoring may be required); and (e) institutional support for curriculum innovation. These contextual factors are not explicitly addressed in the plan but will significantly influence outcomes.

7. Conclusion and Actionable Recommendations

This peer-review critique has examined the Sequential Training Plan for Physics-Informed Machine Learning and Building Performance Simulation through the lens of recent literature (2023-2025), addressing four critical dimensions: Phase 1 timeline feasibility, domain-first strategy validation, curriculum currency, and pedagogical progression logic. The analysis reveals a theoretically sound and ambitious program with strong technical foundations but several implementation challenges requiring attention.

Summary of Key Findings

Strengths:

- Strong technical justification for physics-informed approaches, with empirical evidence of dramatic safety improvements (156-fold reduction in temperature violations) and enhanced data efficiency
- Logical pedagogical progression from domain mastery through AI capabilities to integrated autonomous systems
- Good coverage of core PIML paradigms including modularized neural networks, physics-informed RL, and component-based ML
- Appropriate emphasis on validation and physical consistency checking

Critical Weaknesses:

- Optimistic three-month timeline for Phase 1 without empirical time-to-competency benchmarks
- Lack of controlled pedagogical studies validating domain-first versus integrated learning sequences
- Missing coverage of emerging methods (physics-informed GNNs, neuro-symbolic approaches, co-simulation)

- Insufficient intermediate scaffolding between manual scripting and autonomous agent development
- Limited attention to AI testing, validation, and failure mode analysis

Prioritized Actionable Recommendations

Immediate Priority (Implement Before Next Offering):

1. Establish programming prerequisites or extend Phase 1. Require demonstrated Python proficiency (data structures, pandas, file I/O) before enrollment, or extend Phase 1 to four months with intensive Python bootcamp for students lacking prerequisites.
2. Implement mandatory domain checkpoints with automated validation. Add weekly graded checkpoints in Months 1-2 that automatically validate code correctness, simulation execution, and physical coherence of outputs (temperature bounds, energy balance).
3. Add intermediate scaffolding module between Months 5 and 6. Insert a 2-week "AI Testing and Validation" module covering prompt engineering, unit testing of AI-generated code, constraint validation frameworks, and debugging strategies.
4. Redesign Month 7 to emphasize human-AI collaboration. Reframe the autonomous agent as a human-in-the-loop decision support system with explicit design of oversight mechanisms, rather than fully autonomous operation.

High Priority (Implement Within One Year):

5. Allow AI-assisted learning during Phase 1 with mandatory validation. Permit LLM use for debugging and code assistance in Months 1-2, but require documented prompts, manual code review, unit tests, and physical validation of all AI-generated content.
6. Expand curriculum to include physics-informed GNNs and neuro-symbolic methods. Add 1-week modules in Month 3 covering graph-based building system representations (following Shan et al., 2025) and symbolic regression/KAN methods (following Chen et al., 2024).

7. Create comprehensive prompt engineering curriculum in Month 4. Transform Month 4 from a single debugging exercise into a systematic prompt engineering module with multiple graded exercises covering different task types and evaluation rubrics.

8. Add explicit module on physical constraint validation in Month 6. Require students to build a "Physics Validation Library" that checks AI-generated simulation parameters against domain constraints, using failure cases from Jiang et al. (2024) as motivation.

Medium Priority (Implement Within Two Years):

9. Conduct pedagogical research to validate the domain-first approach. Design quasi-experimental study comparing domain-first versus integrated learning sequences, measuring ability to identify physically implausible outputs, debugging effectiveness, and long-term knowledge retention.

10. Instrument curriculum to collect time-to-competency data. Systematically track student progress through automated logging of time spent, debugging iterations, and checkpoint success rates to enable evidence-based timeline refinement.

11. Expand tool coverage to include co-simulation frameworks. Add 1-week module in Month 2 on EnergyPlus-CONTAM coupling for integrated energy and indoor air quality assessment (following Ashayeri et al., 2024).

12. Create "State-of-the-Art Review" assignment in Month 3. Require students to review one recent PIML method (2024-2025) not covered in curriculum, present to class, and propose building performance application.

Ongoing Priorities:

13. Verify and update all cited sources. Conduct systematic review of all cited methodologies to ensure they are published, peer-reviewed, and accessible; replace any unpublished sources with validated alternatives.

14. Establish curriculum updating mechanism. Create annual review process to incorporate new PIML methods, updated LLM capabilities, and emerging tools, given the rapid evolution of the field documented by Maalek (2024).

15. Develop "AI Failure Case Studies" throughout Phase 2. In each month of Phase 2, analyze real-world AI failures in engineering contexts and require students to propose prevention strategies based on domain knowledge and validation procedures.

Final Assessment

The Sequential Training Plan represents a valuable and innovative approach to preparing civil engineers for AI-enabled building performance simulation. Its domain-first philosophy is well-justified by technical literature demonstrating the critical importance of physics-informed approaches for safe and effective AI deployment in building systems. However, successful implementation will require careful attention to timeline realism, intermediate scaffolding, and continuous curriculum updating to keep pace with rapid advances in both PIML methods and generative AI capabilities.

With the recommended modifications—particularly strengthened prerequisites, mandatory validation checkpoints, intermediate AI testing modules, and expanded coverage of emerging methods—the plan has strong potential to produce graduates who can effectively combine deep domain expertise with cutting-edge AI capabilities. The program's success will ultimately depend on rigorous implementation, ongoing empirical evaluation of learning outcomes, and willingness to iteratively refine the curriculum based on evidence from both pedagogical research and student performance data.

References

- Alphinas, A., Kalaitzakis, M., & Kalliris, G. (2024). Generative AI learning environment for non-computer science engineering students: Coding versus generative AI prompting. *Proceedings of the International Conference on AI Research*, 4(1). <https://doi.org/10.34190/icair.4.1.3220>
- Ashayeri, M., Kampf, J. H., & Nikkho, S. K. (2024). A hybrid physics-based machine learning approach for integrated energy and exposure modeling. In *Building Physics and Sustainable Design* (pp. 47-76). Wiley. <https://doi.org/10.1002/9781394172092.ch3>
- Chen, Y., Hong, T., & Piette, M. A. (2023). Pathway toward prior knowledge-integrated machine learning in engineering. *Building Simulation Conference Proceedings*, 18, 1481-1488. <https://doi.org/10.26868/25222708.2023.1481>

Chen, Y., Hong, T., & Piette, M. A. (2024). Utilizing domain knowledge: Robust machine learning for building energy performance prediction with small, inconsistent datasets. *SSRN Electronic Journal*.

<https://doi.org/10.2139/ssrn.4726883>

Chen, Z., Wang, J., Feng, W., & Yin, H. (2024). Integrating symbolic neural networks with building physics: A study and proposal. *arXiv preprint arXiv:2411.00800*. <https://doi.org/10.48550/arxiv.2411.00800>

Danish, M., Qureshi, M. I., & Sharma, D. (2025). Physics-guided memory network for building energy modeling. *Energy and AI*, 19, 100538. <https://doi.org/10.1016/j.egyai.2025.100538>

Jiang, Z., Risbeck, M. J., Ramamurti, V., Miller, C., Annex, J., & Vrabie, D. (2024). Modularized neural network incorporating physical priors for smart building control: Accuracy or consistency? *arXiv preprint arXiv:2412.02943*.

<https://doi.org/10.48550/arxiv.2412.02943>

Jiang, Z., Risbeck, M. J., Ramamurti, V., Miller, C., Annex, J., & Vrabie, D. (2025). Physics-informed modularized neural network for advanced building control by deep reinforcement learning. *arXiv preprint arXiv:2504.05397*.

<https://doi.org/10.48550/arxiv.2504.05397>

Kuo, H. J., Hsieh, S. H., Guo, S. J., & Chan, C. C. (2023). Physics-informed machine learning framework to model buildings from incomplete information. *Journal of Physics: Conference Series*, 2600(7), 072013.

<https://doi.org/10.1088/1742-6596/2600/7/072013>

Labib, R. (2024). Utilizing physics-informed neural networks to advance daylighting simulations in buildings. *SSRN Electronic Journal*. <https://doi.org/10.2139/ssrn.4741587>

Liu, Y. (2024). Application of large language models in engineering education: A case study of system modeling and simulation courses. *The International Journal of Mechanical Engineering Education*, 52(4), 345-362.

<https://doi.org/10.1177/03064190241272728>

Ma, Z., Ren, H., & Lin, W. (2024). Physics-informed ensemble learning with residual modeling for enhanced building energy prediction. *SSRN Electronic Journal*. <https://doi.org/10.2139/ssrn.4885240>

Maalek, R. (2024). Integrating generative artificial intelligence and problem-based learning into the digitization in construction curriculum. *Buildings*, 14(11), 3642. <https://doi.org/10.3390/buildings14113642>

Saeed, M. H., Cai, J., Risbeck, M. J., Dey, S., & Prakash, A. (2023). Physics-informed model-based reinforcement learning (PI-MBRL) to control building heating systems in low training data regimes. *Proceedings of the 10th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation* (pp. 260-263). ACM.
<https://doi.org/10.1145/3600100.3626270>

Shan, X., Zhou, J., Chang, V. W. C., & Yang, E. H. (2025). Physics-informed and explainable graph neural networks for generalizable urban building energy modeling. *Applied Sciences*, 15(16), 8854. <https://doi.org/10.3390/app15168854>

Zakeri, B. (2025). Generative AI for project-based assessment in energy and sustainability education. *Research Square*.
<https://doi.org/10.21203/rs.3.rs-7402766/v1>

Zhang, Y., Chen, X., & Wang, L. (2024). Leveraging generative AI to enhance engineering education at both low-level and high-level study. *Proceedings of the 2024 ASEE Annual Conference & Exposition*. <https://doi.org/10.18260/1-2--49450>